Unit 4: PL/SQL

1.1 Introduction

PL/SQL is a block structured language that enables developers to combine the power of SQL with procedural statements.All the statements of a block are passed to oracle engine all at once which increases processing speed and decreases the traffic.

Disadvantages of SQL:

- SQL doesn't provide the programmers with a technique of condition checking, looping and branching.
- SQL statements are passed to Oracle engine one at a time which increases traffic and decreases speed.
- SQL has no facility of error checking during manipulation of data.

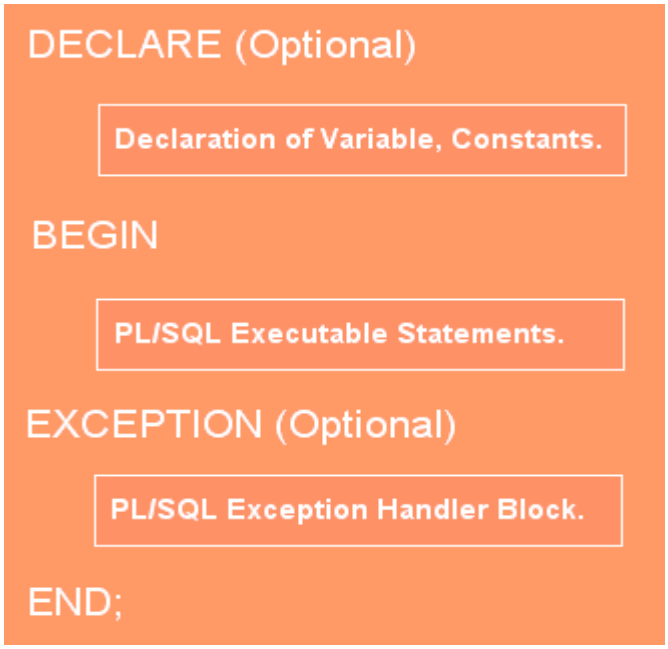
1.2 Advantages of PL/SQL

PL/SQL offers the following advantages:

- Reduces network traffic This one is **great advantages** of PL/SQL. Because PL/SQL nature is **entire block** of SQL statements execute into **oracle engine** all at once so it's main benefit is **reducing** the **network traffic**.
- Procedural language support PL/SQL is a **development tools** not only for data manipulation futures but also provide the conditional checking, looping or branching operations same as like **other programming language**.
- Error handling PL/SQL is dealing with **error handling**, It's permits the smart way **handling the errors** and giving **user friendly** error messages, when the errors are encountered.
- Declare variable PL/SQL gives you control to **declare variables** and access them **within the block**. The declared variables can be used at the time of **query processing**.
- Intermediate Calculation Calculations in PL/SQL done quickly and efficiently without using Oracle engines. This **improves** the transaction performance.
- Portable application Applications are written in PL/SQL are **portable** in any **Operating system**. PL/SQL applications are **independence program** to run any computer.

1.3 Generic PL/SQL Block

A PL/SQL block has a definite structure, which can be divided into sections. The sections of a PL/SQL block are:



DECLARE

Variables and constants are declared, initialized within this section. Once declared, they can be used in SQL statements for data manipulations.

BEGIN

BEGIN block is a procedural statement block which will implement the actual programming logic. This section contains conditional statements (if...else), looping statements (for, while) and Branching Statements (goto), etc.

EXCEPTION

PL/SQL easily detects a user-defined or predefined error condition. PL/SQL is famous for smartly handling errors by giving suitable user-friendly messages. Errors can be rise due to the wrong syntax, bad logical, or not passing validation rules.

END Section

This marks the end of a PL/SQL block.

Example

```
DECLARE
message varchar2(20):= 'Hello, World!';
BEGIN
dbms_output.put_line(message);
END;
/
```

1.4 PL/SQL Character Set

The following characters will be available to you:

- **Uppercase alphabets :** A–Z
- **Lowercase alphabets** : a–z
- **Digits:** 0–9
- **Symbols:** ~ ! @ # $ % * () _ − + = | : ; “ ‘ <> , . ? / ^

Words used in a PL/SQL block are called **LexicalUnits.** Blank spaces can be freely inserted between lexical units in a PL/SQL block. The blank spaces have no effect on the PL/SQL block.

The ordinary symbols used in PL/SQL blocks are:

( ) + - * / <> = ; % ‘ “ [ ] :

Compound symbols used in PL/SQL blocks are:

<> != ~= ^= <= >= := ** .. || <<>>

1.5 Literals

A literals is a numeric value or a character string used to represent itself.

- Numeric Literals
  These can be either integers or floats . If a float is being represented, then the integer part must be separated from the float part by a period.

Example:

050 78 -14 0 +32767

6.6667 0.0 -12.0 3.14159 +7800.00

Character Literals

These are string literals consisting of single characters.

'A' '%' '9' ' ' 'z' '('

String literals

These are represented by one or more legal character and must be enclosed within single quotes . The single quotes character can be represented, by writing it twice in a string literals. This is definitely not same as double quotes .

Example: 'Hello, world!'

Logical(Boolean) literals

These are predetermined constants. The value that can be assigned to this data type are:TRUE,FALSE,NULL.

1.6 PL/SQL data type

The default datatypes that can be declared in PL/SQL are **number(**for storing numeric data), **char** (for storing character data) , **date**(for storing date and time**), Boolean** (for storing TRUE, FALSE OR NULL).

The **%TYPE** data type allows you to declare a variable that is associated with a column in a database table.The Oracle PL/SQL %TYPE attribute allow you to declare a constant, variable, or parameter to be of the same data type as previously declared variable, record, nested table, or database column.

NOT NULL causes creation of a variable or a constant that cannot be assigned a null value.

1.7 Variables

A PL/SQL variable is a placeholder for a value. This value can be a number, date, or text string. You can use variables in your Oracle PL/SQL programs to store values temporarily.
Syntax

- A variable must begin with a character and can be followed by a maximum of 29 other characters.
- Reserved words cannot be used as variable names unless enclosed within double quotes.
- Variable must be separated from each other by at least one space or by a punctuation mark.
- Case is insignificant when declaring variable names.
- A space cannot be used in a variable name.

To create a variable, you use the following syntax:

```
variable_namedatatype [NOT NULL] [:= | DEFAULT initial_value];
```

Example :

```
my_num number := 10;
```

1.8 Logical comparison

PL/SQL supports the comparison between variable and constants in SQL anPL/SQL statements. These comparison , often called **Boolean expression** generally consist of simple expression separated by relation operators (<,>,=,<>,>=,<=) that can be connected by logical operators (AND, OR, NOT). A Boolean expression will always evaluate to TRUE, FALSE or NULL.

1.9 Displaying User Message On The VDU Screen

The **DBMS_OUTPUT** is a built-in package that enables you to display output, debugging information, and send messages from PL/SQL blocks, subprograms, packages, and triggers.

PUT_LINE puts a piece of information in the package buffer followed by an end-of-end line marker. It can also be used to display message. **PUT_LINE** expectsa single parameter of character data type.

To display messages, the SERVEROUTPUT should be set to ON. SERVEROUTPUT is a SQL*Plus environment parameter that displays the information passed as a parameter to the **PUT_LINE** function**.**

Syntax: SET SERVEROUTPUT[ON/OFF]

1.10 Comments

- A single-line comment starts with a double hyphen (--) that can appear anywhere on a line and extends to the end of the line.
- A multi-line comment starts with a slash-asterisk (/*) and ends with an asterisk-slash (*/), and can span multiple lines.

1.11 Control Structures

- Conditional control
- Iterative control
- Sequential control

Conditional control

The IF statement lets you execute a sequence of statements conditionally. That is, whether the sequence is executed or not depends on the value of a condition. There are three forms of IF statements: IF-THEN, IF-THEN-ELSE, and IF-THEN-ELSIF. The CASE statement is a compact way to evaluate a single condition and choose between many alternative actions.

IF-THEN Statement

The simplest form of IF statement associates a condition with a sequence of statements enclosed by the keywords THEN and END IF (not ENDIF), as follows:

```
IF condition1 THEN
   sequence_of_statements1
ELSIF condition2 THEN
   sequence_of_statements2
ELSE
   sequence_of_statements3
END IF;
```

Example:

```
BEGIN
   ...
   IF sales > 50000 THEN
bonus := 1500;
   ELSIF sales > 35000 THEN
bonus := 500;
   ELSE
bonus := 100;
   END IF;
   INSERT INTO payroll VALUES (emp_id, bonus, ...);
END;
```

Iterative Control

LOOP statements let you execute a sequence of statements multiple times. There are three forms of LOOP statements: LOOP, WHILE-LOOP, and FOR-LOOP.

LOOP

The simplest form of LOOP statement is the basic (or infinite) loop, which encloses a sequence of statements between the keywords LOOP and END LOOP, as follows:

```
LOOP
sequence_of_statements
END LOOP;
```

With each iteration of the loop, the sequence of statements is executed, then control resumes at the top of the loop. If further processing is undesirable or impossible, you can use an EXIT statement to complete the loop. You can place one or more EXIT statements anywhere inside a loop, but nowhere outside a loop. There are two forms of EXIT statements: EXIT and EXIT-WHEN.

```
DECLARE
        i NUMBER:=0;
BEGIN
LOOP
i:=i+2;
      EXIT WHEN i>10;
END LOOP;
END;
```

WHILE-LOOP

The WHILE-LOOP statement associates a condition with a sequence of statements enclosed by the keywords LOOP and END LOOP, as follows:

```
Syntax:
WHILE condition
LOOP
sequence_of_statements
```

```
END LOOP;
```

Example :

```
WHILE total <= 25000 LOOP
   ...
   SELECT sal INTO salary FROM empWHERE ...
total := total + salary;
END LOOP;
```

FOR-LOOP

Whereas the number of iterations through a `WHILE` loop is unknown until the loop completes, the number of iterations through a `FOR` loop is known before the loop is entered. `FOR` loops iterate over a specified range of integers. The range is part of an *iteration scheme*, which is enclosed by the keywords `FOR` and `LOOP`. A double dot (. .) serves as the range operator. The syntax follows:

```
FOR variable IN [REVERSE] lower_bound..higher_bound
LOOP
sequence_of_statements
END LOOP;
```

```
Example
FOR j IN 1..3 LOOP
dates(j*k) := SYSDATE;     -- multiply loop counter by increment
   END LOOP;
```

Chapter 2: PL/SQL Transaction

2.1 Cursor

To execute SQL statements, a work area is used by the Oracle engine for its internal processing and storing the information. This work area is private to SQL's operations and is called a **cursor.**

The stored in the cursor is called the **active data set.** Size of the cursor in memory is the size required to hold the number of rows in the **active data set.**

Types of Cursors

- **Implicit Cursors**

Implicit cursors are automatically created by Oracle whenever an SQL statement is executed, when there is no explicit cursor for the statement. Programmers cannot control the implicit cursors and the information in it.

The following table provides the description of the most used attributes –

S.No	Attribute & Description
1	**%FOUND** Returns TRUE if an INSERT, UPDATE, or DELETE statement affected one or more rows or a SELECT INTO statement returned one or more rows. Otherwise, it returns FALSE.
2	**%NOTFOUND** The logical opposite of %FOUND. It returns TRUE if an INSERT, UPDATE, or DELETE statement affected no rows, or a SELECT INTO statement returned no rows. Otherwise, it returns FALSE.
3	**%ISOPEN** Always returns FALSE for implicit cursors, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.
4	**%ROWCOUNT** Returns the number of rows affected by an INSERT, UPDATE, or DELETE statement, or returned by a SELECT INTO statement.

Example: The following program will update the table and increase the salary of each customer by 500 and use the **SQL%ROWCOUNT** attribute to determine the number of rows affected –

```
DECLARE
total_rows number(2);
BEGIN
   UPDATE customers
   SET salary = salary +500;
   IF sql%notfound THEN
dbms_output.put_line('no customers selected');
   ELSIF sql%found THEN
total_rows:=sql%rowcount;
dbms_output.put_line(total_rows||' customers selected ');
END IF;
END;
/
```

- **Explicit Cursors**

Explicit cursors are programmer-defined cursors for gaining more control over the **context area**. An explicit cursor should be defined in the declaration section of the PL/SQL Block. It is created on a SELECT Statement which returns more than one row.

Syntax for creating an explicit cursor:

CURSOR cursor_name IS select_statement;

Working with an explicit cursor includes the following steps −

- Declaring the cursor for initializing the memory
- Opening the cursor for allocating the memory
- Fetching the cursor for retrieving the data
- Closing the cursor to release the allocated memory

Declaring the Cursor

Declaring the cursor defines the cursor with a name and the associated SELECT statement. For example –

```
CURSOR c_customers IS
   SELECT id, name, address FROM customers;
```

Opening the Cursor

Opening the cursor allocates the memory for the cursor and makes it ready for fetching the rows returned by the SQL statement into it. For example, we will open the above defined cursor as follows −

```
OPEN c_customers;
```

Fetching the Cursor

Fetching the cursor involves accessing one row at a time. For example, we will fetch rows from the above-opened cursor as follows −

```
FETCH c_customers INTO c_id,c_name,c_addr;
```

Closing the Cursor

Closing the cursor means releasing the allocated memory. For example, we will close the above-opened cursor as follows −

```
CLOSE c_customers;
```

Example

```
DECLARE
c_idcustomers.id%type;
c_namecustomers.name%type;
c_addrcustomers.address%type;
   CURSOR c_customersis
      SELECT id, name, address FROM customers;
BEGIN
   OPEN c_customers;
   LOOP
   FETCH c_customersintoc_id,c_name,c_addr;
      EXIT WHEN c_customers%notfound;
dbms_output.put_line(c_id||' '||c_name||' '||c_addr);
END LOOP;
   CLOSE c_customers;
END;
/
```

When the above code is executed at the SQL prompt, it produces the following result –

```
1 Ramesh Ahmedabad
2 Khilan Delhi
3 kaushik Kota
4 Chaitali Mumbai
5 Hardik Bhopal
6 Komal MP

PL/SQL procedure successfully completed.
```

CURSOR FOR Loop

This Oracle tutorial explains how to use the **CURSOR FOR LOOP** in Oracle with syntax and examples.

You would use a CURSOR FOR LOOP when you want to fetch and process every record in a cursor. The CURSOR FOR LOOP will terminate when all of the records in the cursor have been fetched.

Syntax

The syntax for the CURSOR FOR LOOP in Oracle/PLSQL is:

```
FOR record_index in cursor_name
LOOP
{...statements...}
END LOOP;
```

Example

DECLARE

CURSOR c_product ISSELECTproduct_name, list_priceFROMproducts;

BEGIN

FOR r_product IN c_product

LOOP

dbms_output.put_line(r_product.product_name || ': $' || r_product.list_price);

END LOOP;

END;

PL/SQL Parameterized Cursor

- PL/SQL Parameterized cursor pass the parameters into a cursor and use them in to query.
- PL/SQL Parameterized cursor define only datatype of parameter and not need to define it's length.
- Default values is assigned to the Cursor parameters. and scope of the parameters are locally.
- Parameterized cursors are also saying static cursors that can passed parameter value when cursor are opened.

Example:

```
DECLARE
cursorc1(id number)isselect*fromemp_informationwhereemp_no=id;
tmpemp_information%rowtype;
BEGIN
OPENc(4);
FORtmpINc(4)LOOP
```

```
dbms_output.put_line('EMP_No:    '||tmp.emp_no);
dbms_output.put_line('EMP_Name:  '||tmp.emp_name);
dbms_output.put_line('EMP_Dept:  '||tmp.emp_dept);
dbms_output.put_line('EMP_Salary:'||tmp.emp_salary);
ENDLoop;
CLOSEc;
END;
/
```

Chapter 3: PL/SQL database object

Procedures and Functions are the subprograms which can be created and saved in the database as database objects. They are logically grouped set of SQL and PL/SQL statements that perform a specific task. They can be called or referred inside the other blocks also.

Parts of a PL/SQL Subprogram

Each PL/SQL subprogram has a name, and may also have a parameter list. Like anonymous PL/SQL blocks, the named blocks will also have the following three parts –

S.No	Parts & Description
1	**Declarative Part** It is an optional part. However, the declarative part for a subprogram does not start with the DECLARE keyword. It contains declarations of types, cursors, constants, variables, exceptions, and nested subprograms. These items are local to the subprogram and cease to exist when the subprogram completes execution.
2	**Executable Part** This is a mandatory part and contains statements that perform the designated action.
3	**Exception-handling** This is again an optional part. It contains the code that handles run-time errors.

How does the oracle engine create Stored Procedure or Function

Whenever a block of code for stored procedure or function is written it is then, they are automatically compiled by the oracle engine. During compilation if any error occurs, we get a message on the screen saying that the procedure or function is created with compilation errors but actual error is not displayed.

In order to find out the compilation errors following statement can be executed:

```
SELECT*from USER_ERRORS;
```

Once it is compiled, it is then stored by the oracle engine in the database as a **database object**.

When a procedure or function is invoked, the oracle engine loads the compiled procedure or function in a memory area called the System Global Area(SGA). This allows the code to be executed quickly. Once loaded in the SGA other users also access the same procedure or function if they have been granted permission to do so.

Advantages of using Procedure or Function

1. Improves Database Performance

- Compilation is automatically done by oracle engine.
- Whenever the calling of procedure or function is done ,the oracle engine loads the compiled code into a memory area called System Global Area(SGA) due to which execution becomes faster.

2. Provides Reusability and avoids redundancy

- The same block of code for procedure or function can be called any number of times for working on multiple data.
- Due to which number of lines of code cannot be written repeatedly.

3. Maintains Integrity

- Integrity means accuracy. Use of procedure or function ensures integrity because they are stored as database objects by the oracle engine.

4. Maintains Security

- Use of stored procedure or function helps in maintaining the security of the database as access to them and their usage can be controlled by granting access/permission to users while the permission to change or to edit or to manipulate the database may not be granted to users.

5. Saves Memory

- Stored procedure or function have shared memory. Due to which it saves memory as a single copy of either a procedure or a function can be loaded for execution by any number of users who have access permission.

Difference between Stored procedure and Function

Stored Procedure	Function
May or may not returns a value to the calling part of program.	Returns a value to the calling part of the program.
Uses IN, OUT, IN OUT parameter.	Uses only IN parameter.
Returns a value using " OUT" parameter.	Returns a value using "RETURN".
Does not specify the datatype of the value if it is going to return after a calling made to it.	Necessarily specifies the datatype of the value which it is going to return after a calling made to it.
Cannot be called from the function block of code.	Can be called from the procedure block of code.

Syntax for creating Stored Procedure

```
CREATEORREPLACEPROCEDURE<procedure_name>

(<variable_name>IN/OUT/INOUT<datatype>,..

)IS/AS

variable/constant declaration;

BEGIN

     -- PL/SQL subprogram body;

EXCEPTION

     -- Exception Handling block ;

END;
```

Here,

- **procedure_name** is for procedure's name and **variable_name** is the variable name for variable used in the stored procedure.
- CREATE or REPLACE PROCEDURE is a keyword used for specifying the name of the procedure to be created.
- BEGIN, EXCEPTION and END are keywords used to indicate different sections of the procedure being created.
- IN/OUT/IN OUT are parameter modes.
- **IN** mode refers to **READ ONLY mode** which is used for a variable by which it will accept the value from the user. It is the default parameter mode.
- **OUT** mode refers to **WRITE ONLY mode** which is used for a variable that will return the value to the user.
- **IN OUT** mode refers to **READ AND WRITE mode** which is used for a variable that will either accept a value from the user or it will return the value to the user.
- You can simply use END statement to end the procedure definition.

Example 1:

```
create or replace procedure "INSERTUSER"
(id IN NUMBER,
name IN VARCHAR2)
is
begin
insert into user values(id,name);
end;
/
```

Example2:

```
DECLARE
a number;
b number;
c number;
PROCEDURE findMin(x IN number, y IN number, z OUT number) IS
BEGIN
  IF x < y THEN
z:= x;
  ELSE
z:= y;
  END IF;
END;
BEGIN
a:= 23;
b:= 45;
findMin(a, b, c);
dbms_output.put_line(' Minimum of (23, 45) : ' || c);
END;
/
```

Example 3: Use of **in out**datatype

```
DECLARE
```

```
a number;
PROCEDURE squareNum(x IN OUT number) IS
BEGIN
x := x * x;
END;
BEGIN
a:= 23;
squareNum(a);
dbms_output.put_line(' Square of (23): ' || a);
END;
/
```

Syntax for creating Functions in PL/SQL

```
CREATEORREPLACEFUNCTION<function_name>

(<variable_name>IN<datatype>,

<variable_name>IN<datatype>,...)

RETURN<datatype>IS/AS

variable/constant declaration;

BEGIN

     -- PL/SQL subprogram body;

EXCEPTION

     -- Exception Handling block ;

END<function_name>;
```

- *function-name* specifies the name of the function.
- [OR REPLACE] option allows the modification of an existing function.
- The optional parameter list contains name, mode and types of the parameters. IN represents the value that will be passed from outside of the function.
- The function must contain a **return** statement.
- The *RETURN* clause specifies the data type you are going to return from the function.
- *function-body* contains the executable part.
- The AS keyword is used instead of the IS keyword for creating a standalone function.

Example:

```
CREATE OR REPLACE FUNCTION totalCustomers
RETURN number IS
total number(2):=0;
BEGIN
   SELECT count(*)into total
   FROM customers;

   RETURN total;
END;
/
```

Example:

```
DECLARE

a number;

b number;

c number;

FUNCTION findMax(x IN number, y IN number)

RETURN number

IS

z number;
```

```
BEGIN
  IF x > y THEN
z:= x;    ELSE
    Z:= y;
  END IF;
  RETURN z;
END;
BEGIN
a:= 23;
b:= 45;
c := findMax(a, b);
dbms_output.put_line(' Maximum of (23,45): ' || c);
END;
/
```

Executing a Standalone Procedure/function

A standalone procedure/function can be called in two ways −

- Using the **EXECUTE** keyword

Example:EXECUTE greetings;//greeting is the subprogramname.

- Calling the name of the procedure from a PL/SQL block

Example

```
BEGIN
greetings;
END;
/
```

Deleting procedures/ functions

Syntax:

DROP PROCEDUREprocedure_name;

DROP FUNCTIONfunction_name;

Example:

DROP PROCEDUREgreeting;

DROP FUNCTIONfindmin;

PACKAGES

What is a Package?

A package is a schema object that groups logically related PL/SQL types, variables, constants, subprograms, cursors, and exceptions. A package is compiled and stored in the database, where many applications can share its contents.

Parts of a package:

1. Package specification
2. Package body or definition

A package has 2 components one is specification, which declares the public items that can be referenced from outside the package.

Second one is body. The body must define queries for public cursors and code for public subprograms.

Why use packages?

- Each package is easily understood and the interface between packages are simple, clear and well defined.
- Packages allow granting of privileges efficiently
- A package's public variable and cursor persist for the duration of the session. Therefore all cursors and procedure that execute in this environment can share them
- Packages let you share your interface information in the package specification, and hide the implementation details in the package body.
- Package public variables and cursors can persist for the life of a session. They can be shared by all subprograms that run in the environment. They let you maintain data across transactions without storing it in the database.

- Packages offer several advantages: modularity, easier application design, information hiding, added functionality, and better performance. Packages let you encapsulate logically related types, items, and subprograms in a named PL/SQL module. Each package is easy to understand, and the interfaces between packages are simple, clear, and well defined.

Syntax of package specification:

```
CREATE PACKAGE package_name AS
  PROCEDURE procedure_name;
END cust_sal;
/
```

Example

```
CREATE PACKAGE emp_sal AS
  PROCEDURE find_sal(e_id employees.id%type);
END emp_sal;
/
```

Syntax of body or definition:

```
CREATE OR REPLACE PACKAGE BODY package_name AS
  PROCEDURE procedure_name IS
    //procedure body
  END procedure_name;
END package_name;
/
```

Example:

```
CREATE OR REPLACE PACKAGE BODY emp_sal AS
  PROCEDURE find_sal(e_id employees.id%TYPE) IS
  e_sal employees.salary%TYPE;
  BEGIN
    SELECT salary INTO e_sal  FROM employees
    WHERE id = e_id;
    dbms_output.put_line('Salary: '|| e_sal);
  END find_sal;
END emp_sal;
/
```

Invoking packages

Syntax:

PackageName.Subprogram_Name;

Procedures defined in the above package can be executed as follows

```
execute emp_sal.find_sal('S001');
```

OR

call emp_sal.find_sal('S001');

Exception Handling in PL/SQL

An exception is an error which disrupts the normal flow of program instructions. PL/SQL provides us the exception block which raises the exception thus helping the programmer to find out the fault and resolve it.

There are two types of exceptions defined in PL/SQL

- User defined exception.
- System defined exceptions**.**

Syntax to write an exception

WHEN exception **THEN**
statement;

System defined exceptions:

These exceptions are predefined in PL/SQL which get raised WHEN certain **database rule is violated.**

System-defined exceptions are further divided into two categories:

1. Named system exceptions.
2. Unnamed system exceptions.

- **Named system exceptions:** They have a predefined name by the system like ACCESS_INTO_NULL, DUP_VAL_ON_INDEX, LOGIN_DENIED etc. the list is quite big.
  So we will discuss some of the most commonly used exceptions:
  - **NO_DATA_FOUND**: It is raised WHEN a SELECT INTO statement returns *no* rows.
  - **TOO_MANY_ROWS**:It is raised WHEN a SELECT INTO statement returns *more* than one row.
  - VALUE_ERROR:This error is raised WHEN a statement is executed that resulted in an arithmetic, numeric, string, conversion, or constraint error. This error mainly results from programmer error or invalid data input.
  - ZERO_DIVIDE = raises exception WHEN dividing with zero.

Example:

```
DECLARE
temp varchar(20);
BEGIN
```

```
SELECT g_id into temp from geeks where g_name='Sharadh';
exception
WHEN no_data_found THEN
   dbms_output.put_line('ERROR');
   dbms_output.put_line('there is no name as Sharadh');
end;
```

Unnamed system exceptions:
Oracle doesn't provide name for some system exceptions called unnamed system exceptions. These exceptions *don't* occur frequently. These exceptions have two parts *code and an associated message*.
The way to handle to these exceptions is to *assign name* to them using **Pragma EXCEPTION_INIT**
Syntax:

```
PRAGMA EXCEPTION_INIT(exception_name, -error_number);
```

error_number are pre-defined and have negative integer range from -20000 to -20999.

Example:

```
DECLARE
exp exception;
pragma exception_init (exp, -20015);
n int:=10;

BEGIN
FOR i IN 1..n LOOP
        dbms_output.put_line(i*i);
                IF i*i=36 THEN
                        RAISE exp;
                END IF;
END LOOP;
```

```
EXCEPTION
WHEN exp THEN
        dbms_output.put_line('Welcome to Sharada');

END;
```

OUTPUT

Output:

```
1
4
9
16
25
36
Welcome to Sharada
```

User defined exceptions:

This type of users can create their own exceptions according to the need and to raise these exceptions explicitly ***raise*** command is used.

Syntax:

```
DECLARE
<exception_name> EXCEPTION;
BEGIN
<Execution block>
EXCEPTION
WHEN <exception_name> THEN
<Handler>
END;
```

Example:

```
DECLARE
    EXP1 EXCEPTION;
```

```
BEGIN
    IF S_id <0 THEN
    RAISE EXP1;
    ELSE   SELECT * FROM STUDENT WHERE ID:=S_id;
EXCEPTION
   WHEN EXP1 THEN
  dbms_output.putline(‘enter proper id’);
END;
/
```